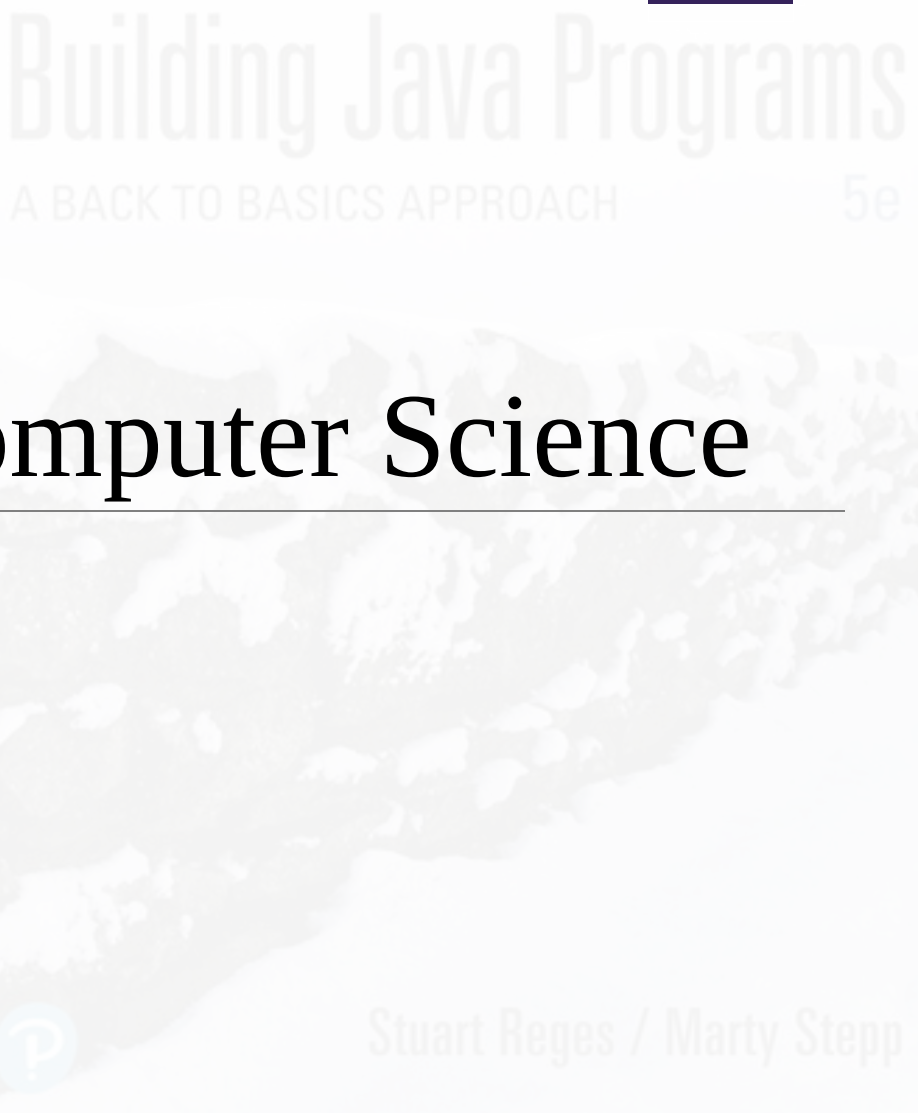


Fundamentals of Computer Science II

09 - Tree (Basic)

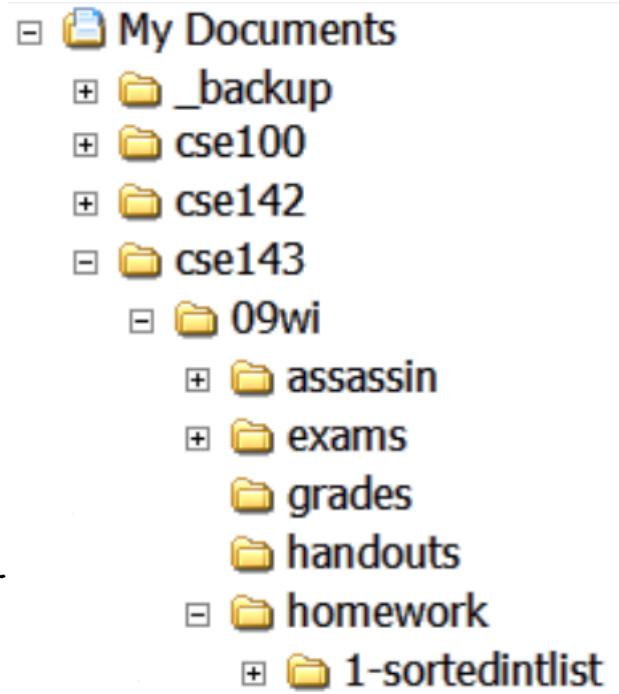
Instructor: Varik Hoang

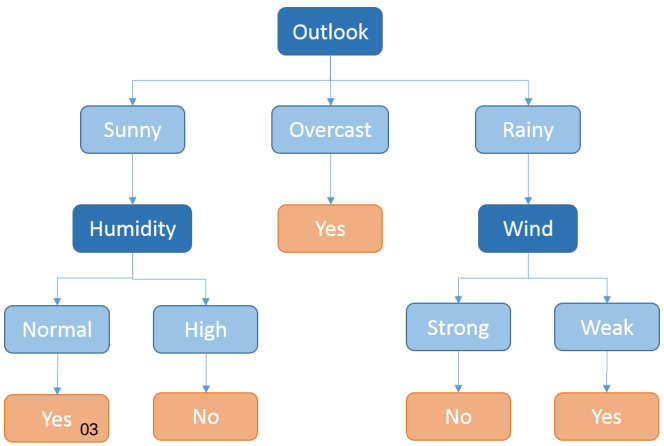
Objectives

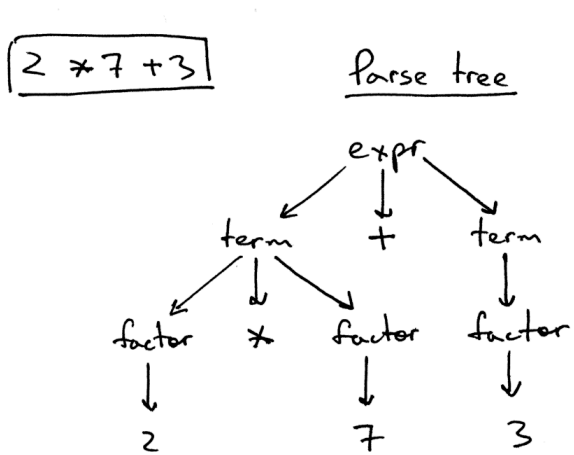
- Binary Tree Basics
- Tree Traversals
- Common Tree Operations
- Binary Search Trees

Trees in Computer Science

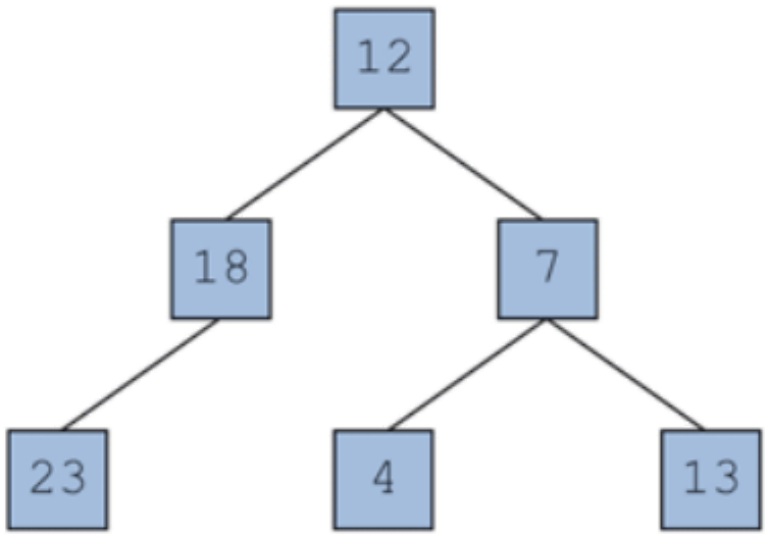
- A list of applications uses tree structure:
  - Folders/files on a computer
  - Family genealogy
  - Organizational charts
  - Decision Trees in Artificial Intelligence
  - Parse Tree in compilers



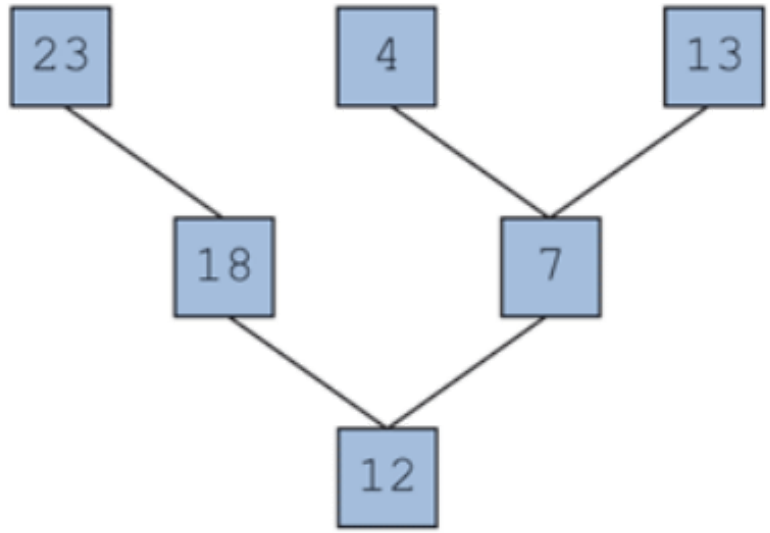




Binary Tree Basics



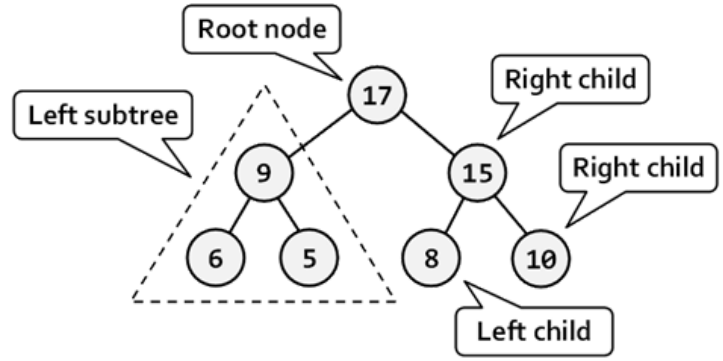
Computer Science Tree



Real-life Tree

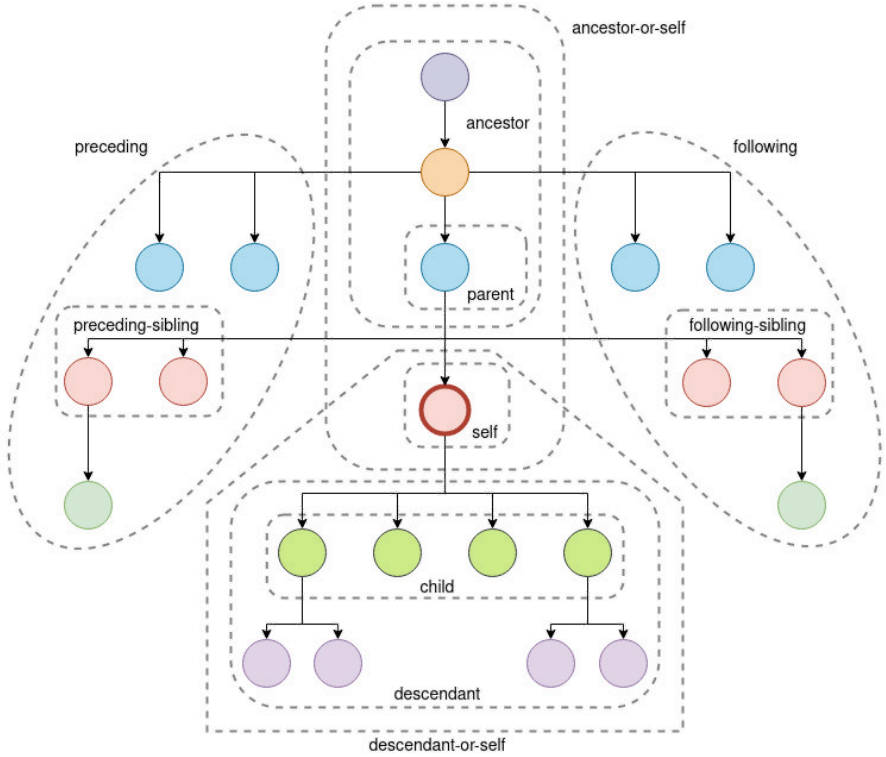
Binary Tree Terminology

- A binary tree is either
  - An **empty tree** or
  - A root node refers to two subtrees **recursively**
- **Root** is a node at the top of a binary tree.
- **Leaf** is a node that has **two** *empty* subtrees.
- **Branch** is a node that has **at least one** *nonempty* subtrees.
- The **edge** is the **connecting link** between **any two nodes**
- Relationships
  - The **parent** node is **predecessor** of any node
  - The **child** node is **descendant** of any node
  - The **sibling** node has the **same parent** to the current node



Binary Tree Terminology (cont.)

- **Ancestor** of a node is
  - Parent of the node
  - Parent of some ancestor of node
- **Descendant** of a node is
  - Child of the node
  - Child of some descendant of node

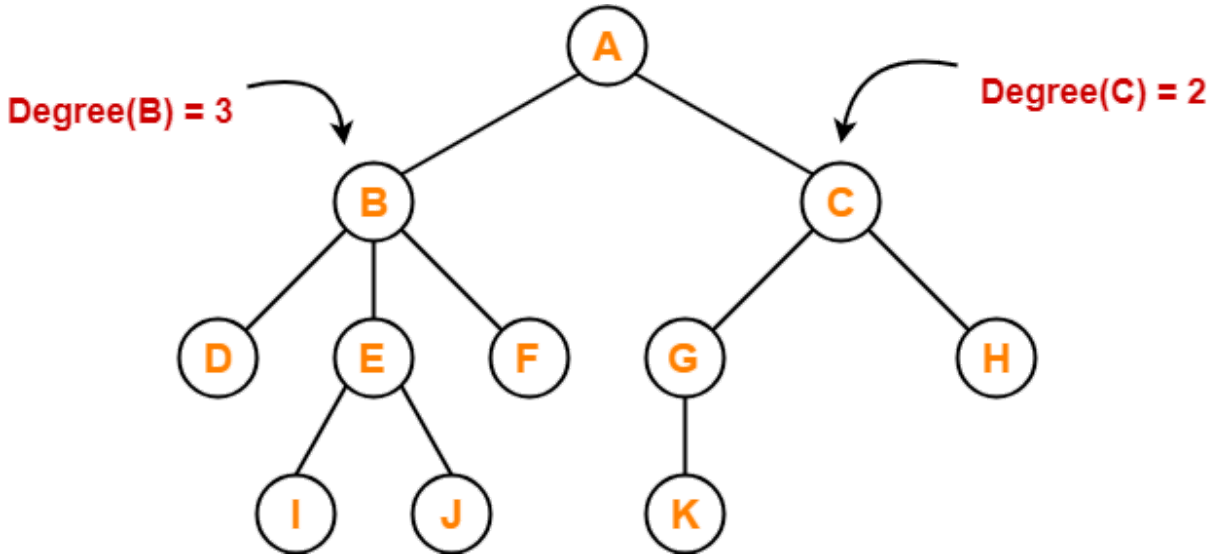


https://jrebecchi.github.io/xpath-helper/xpath-axes.html

Tree Structure - Degree

— **Degree of a node** is the total number of children of that node.

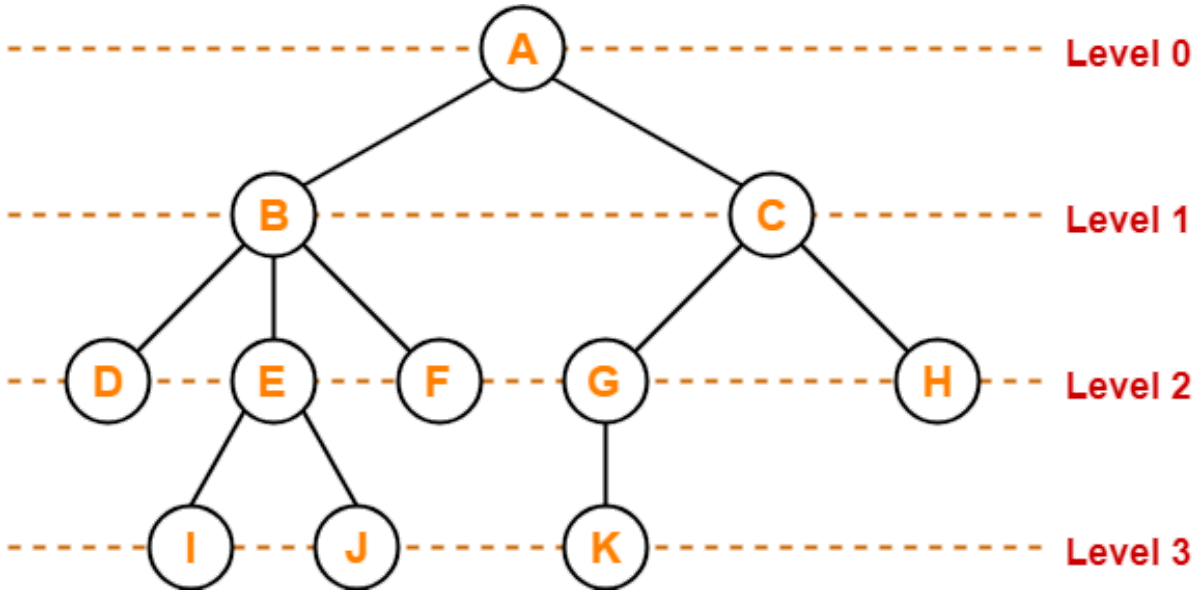
— **Degree of a tree** is the highest degree of a node among all nodes in the tree.



Tree Structure - Level

— **Level of a tree** is each step from top to bottom

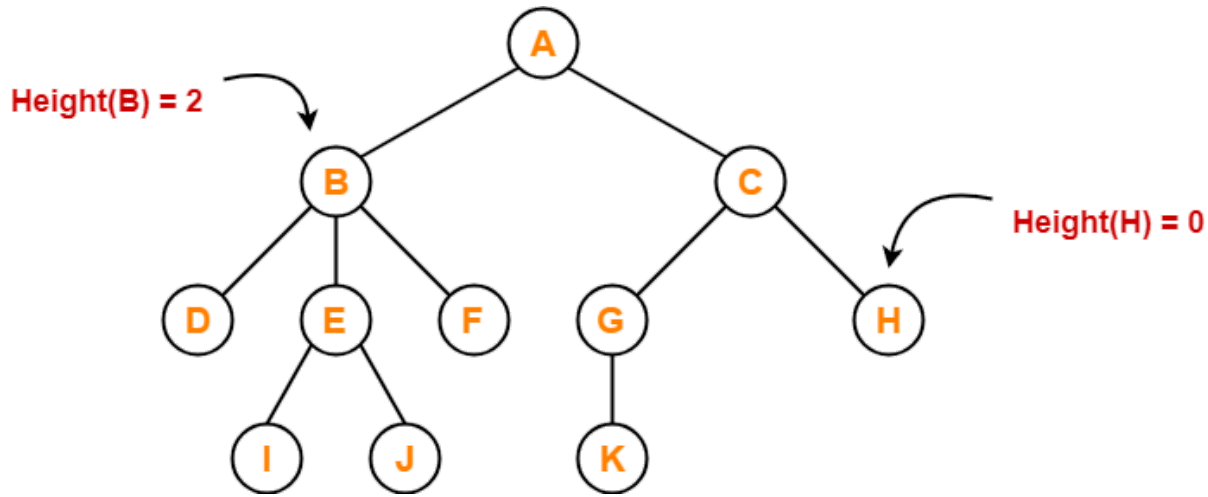
— The **level count** starts with 0.



Tree Structure - Height

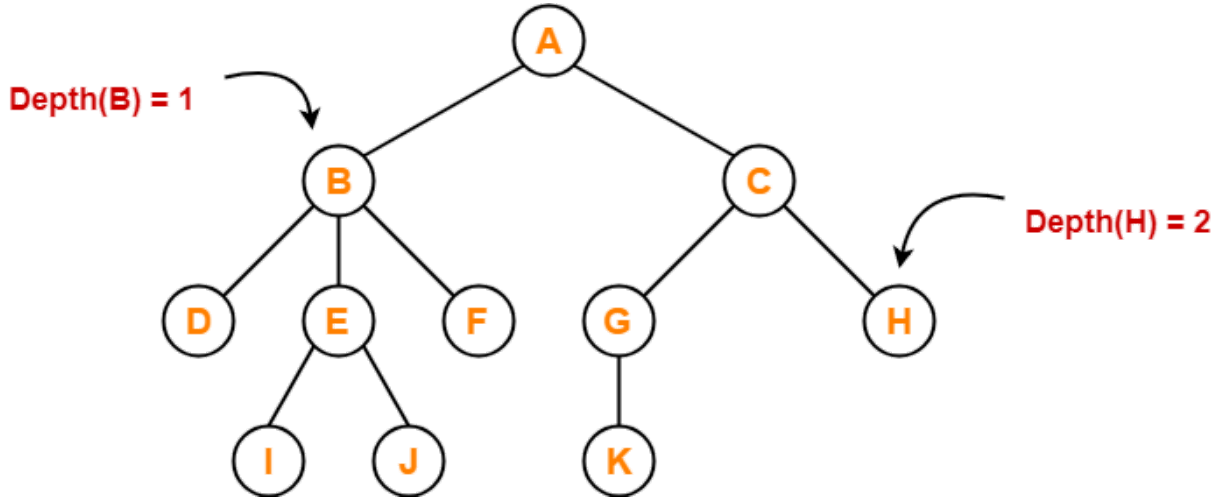
— **Height of a node** is the number of edges that lies on the longest path

— **Height of a tree** is the height of root node. **Height** of all leaf nodes is 0.



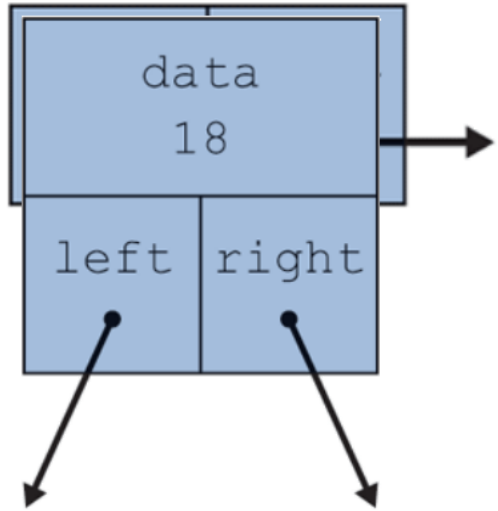
Tree Structure - Depth

- **Depth of a node** is number of edges from root node to a particular node
- **Depth of a tree** is number of edges from root node to a leaf node in the longest path
- **Depth** of the root node is 0



Node

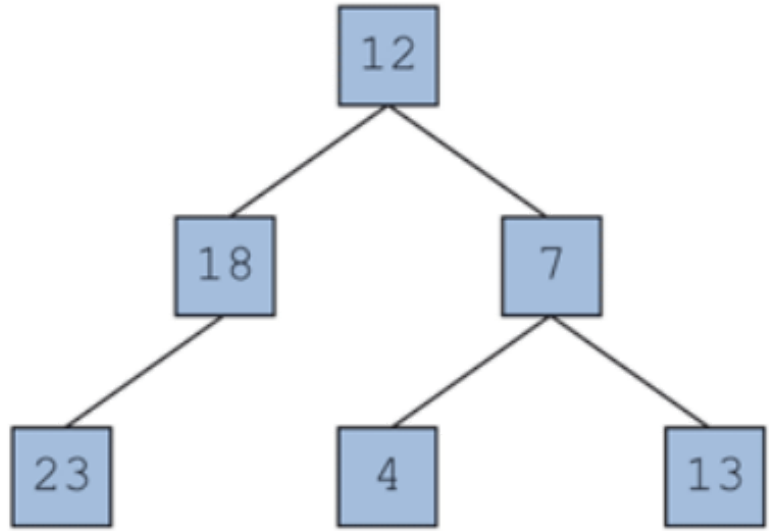
- A single element of a structure such as a binary tree
- Each node contains:
  - one data *value*
  - one *reference* to the *left* node
  - one *reference* to the *right* node




```
class TreeNode
{
    int data;
    TreeNode left;
    TreeNode right;
}
...
TreeNode myNode = new TreeNode();
myNode.data = 18;
myNode.left = null;
myNode.right = null;
```

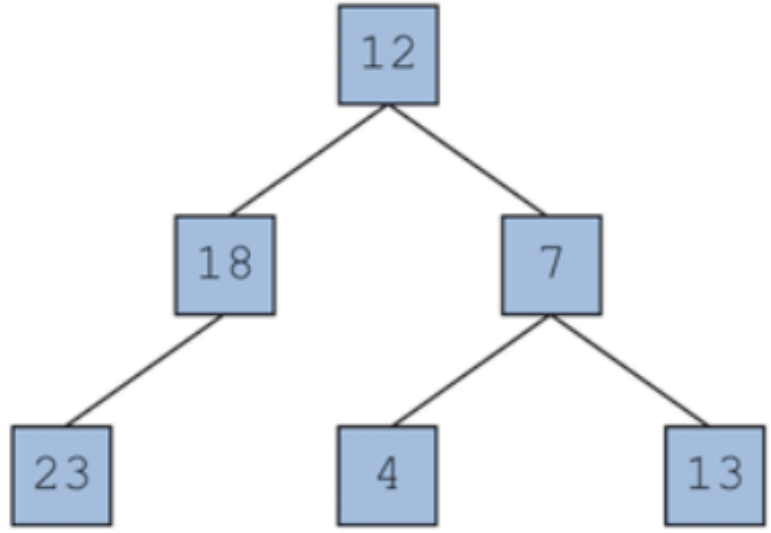
Node (example)

```
class TreeNode
{
    int data;
    TreeNode left;
    TreeNode right;
}
...
TreeNode myTree = new TreeNode();
myTree.data = 12;

myTree.left = new TreeNode();
myTree.left.data = 18;

myTree.left.left = new TreeNode();
myTree.left.left.data = 23;

myTree.right = new TreeNode();
myTree.right.data = 7;

myTree.right.left = new TreeNode();
myTree.right.left.data = 4;

myTree.right.right = new TreeNode();
myTree.right.right.data = 13;
```



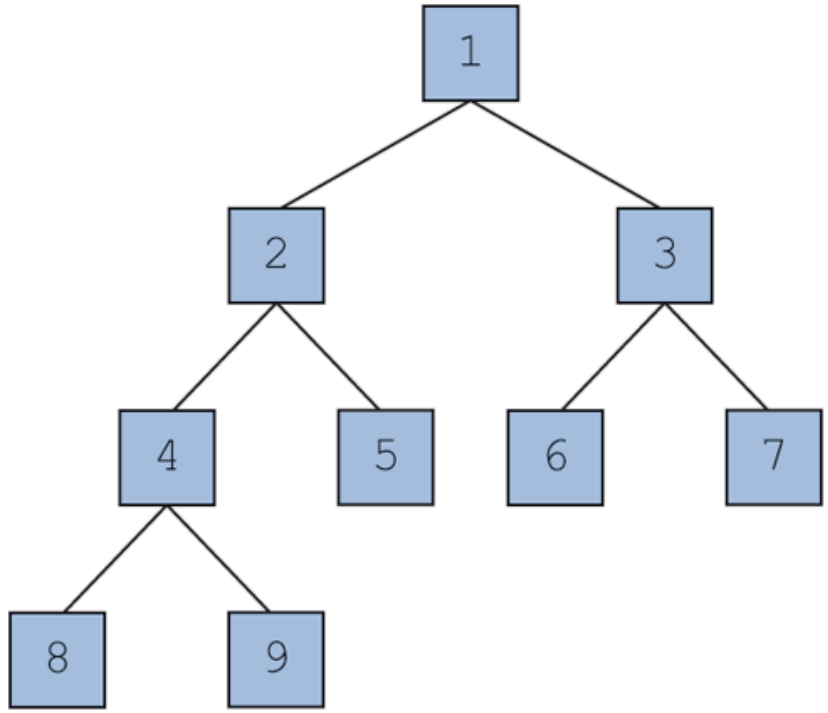
TreeNode with Constructors

```
class TreeNode
{
    ...

    public TreeNode(int data)
    {
        this.data = data;
        this.left = null;
        this.right = null;
    }

    public TreeNode(int data, TreeNode left, TreeNode right)
    {
        this.data = data;
        this.left = left;
        this.right = right;
    }
}
...
TreeNode myNode = new TreeNode(12);
myNode.left = new TreeNode(18, new TreeNode(23), null);
myNode.right = new TreeNode(7, new TreeNode(4)
                               new TreeNode(13));
```



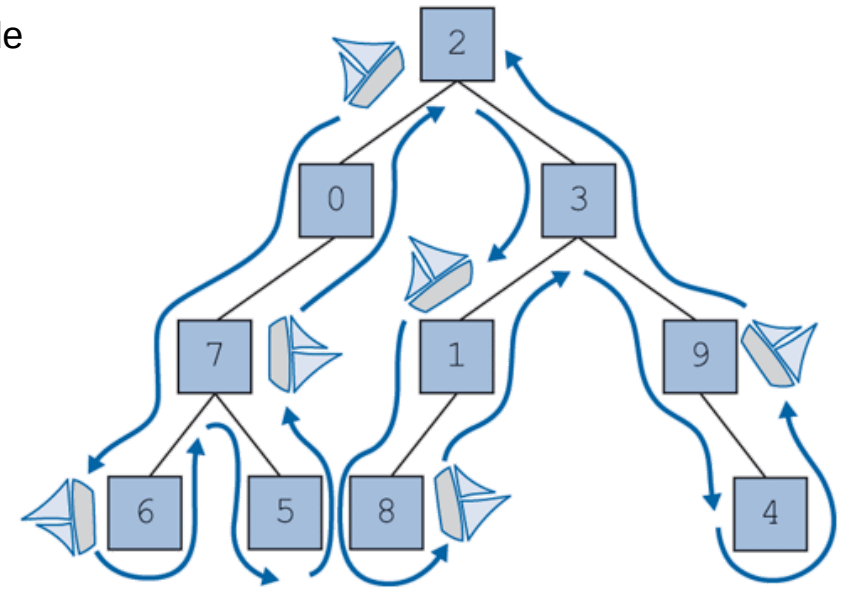
Constructing and Viewing a Tree

```
public class BinaryTree
{
    private TreeNode root;

    public BinaryTree()
    {
        this.root = buildTree(9, 1);
    }

    private TreeNode buildTree(int max, int current)
    {
        // base case
        if (current > max)
            return null;

        // recursive case
        TreeNode left = buildTree(max, 2 * current);
        TreeNode right = buildTree(max, 2 * current + 1);
        return new TreeNode(left, right);
    }

    class TreeNode { ... }
}
```



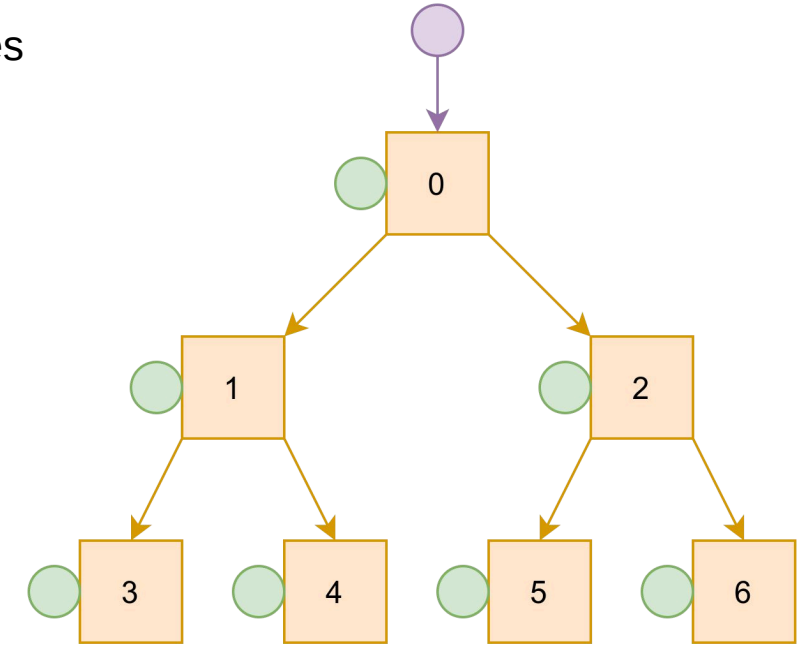
Binary Tree - Traversals

- **Traversal** is an examination of the elements of a tree
- A pattern used in many tree algorithms and methods. Common orderings for traversals
  - **Pre-order**: process node, then its left/right subtrees
  - **In-order**: process left subtree, then node, then right
  - **Post-order**: process left/right subtrees, then node



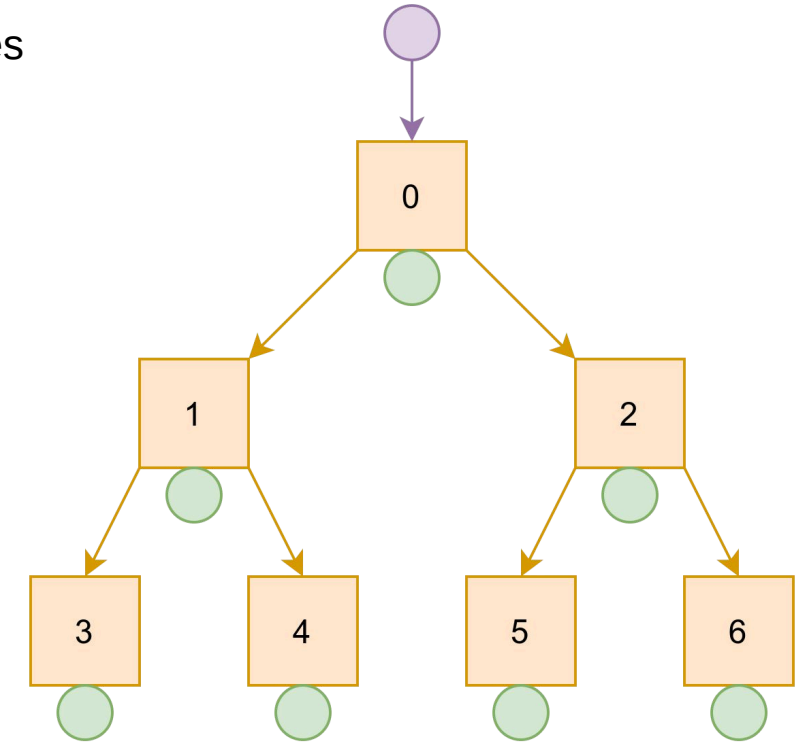
Binary Tree - Pre-order Traversal

- Draw (**green**) circles **on the left** of each node
- Draw curve **following the path** of the tree
- Write down node when **hit** the **green** circles



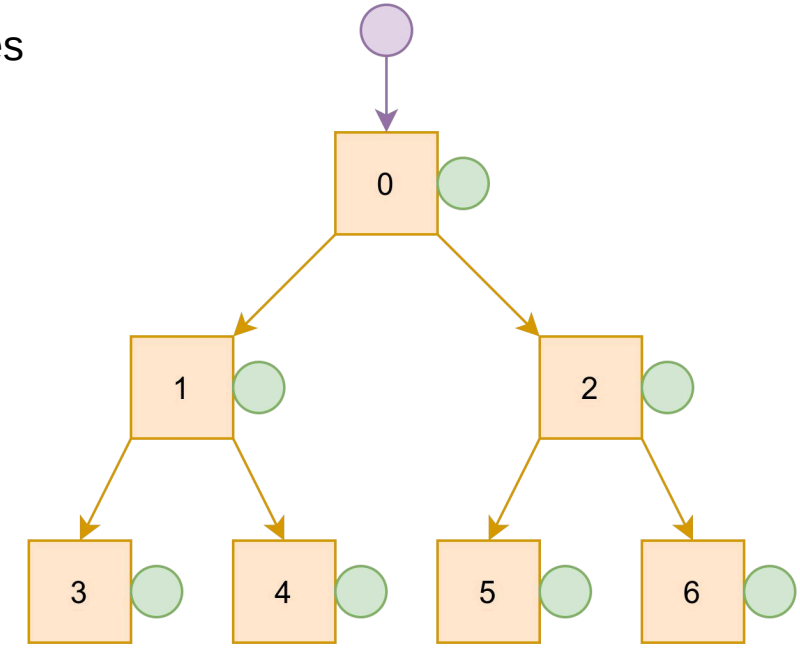
Binary Tree - In-order Traversal

- Draw (**green**) circles **at the bottom** of each node
- Draw curve **following the path** of the tree
- Write down node when **hit** the **green** circles



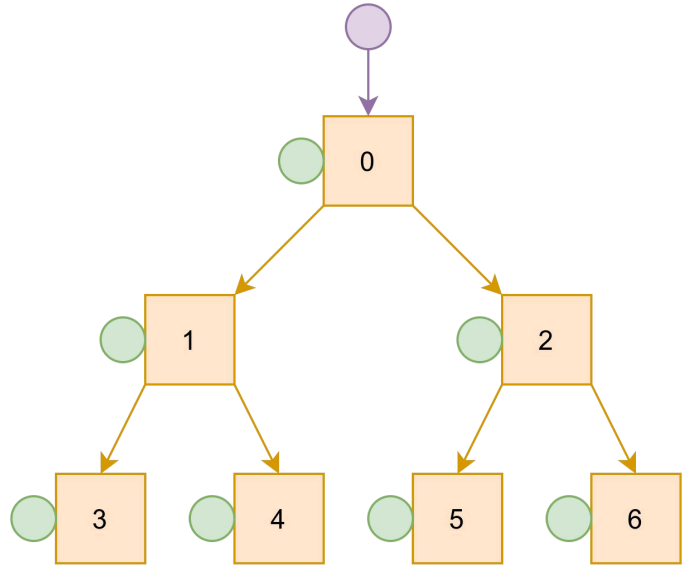
Binary Tree - Post-order Traversal

- Draw (**green**) circles **on the right** of each node
- Draw curve **following the path** of the tree
- Write down node when **hit** the **green** circles



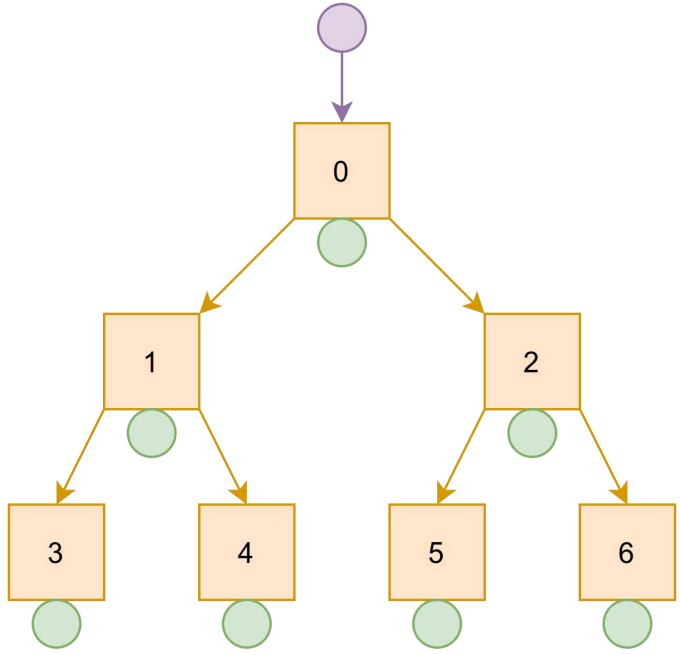
Binary Tree - Pre-order Traversal Implementation

```
public class BinaryTree
{
    private TreeNode root;
    ...
    public void print()
    {
        print(root);
    }

    private void print(TreeNode current)
    {
        // base case
        if (current == null)
            return;

        // recursive case
        System.out.print(current.data + " ");
        print(current.left);
        print(current.right);
    }

    class TreeNode { ... }
}
```



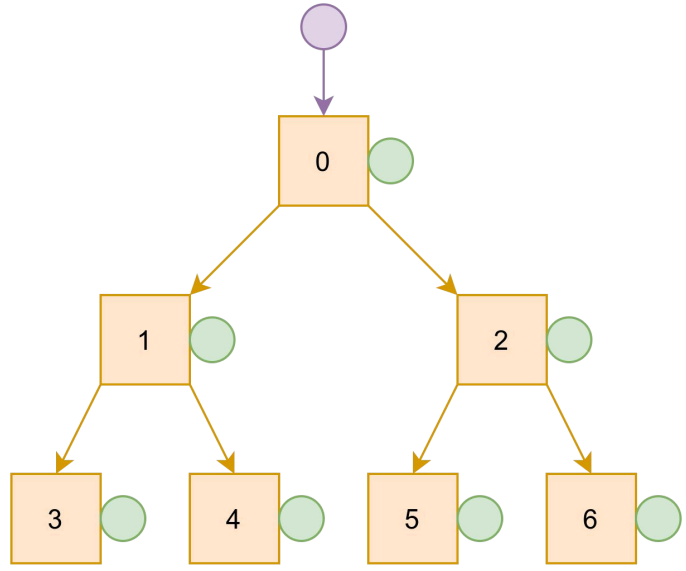
Binary Tree - In-order Traversal Implementation

```
public class BinaryTree
{
    private TreeNode root;
    ...
    public void print()
    {
        print(root);
    }

    private void print(TreeNode current)
    {
        // base case
        if (current == null)
            return;

        // recursive case
        print(current.left);
        System.out.print(current.data + " ");
        print(current.right);
    }

    class TreeNode { ... }
}
```



Binary Tree - Post-order Traversal Implementation

```
public class BinaryTree
{
    private TreeNode root;
    ...
    public void print()
    {
        print(root);
    }

    private void print(TreeNode current)
    {
        // base case
        if (current == null)
            return;

        // recursive case
        print(current.left);
        print(current.right);
        System.out.print(current.data + " ");
    }

    class TreeNode { ... }
}
```



Binary Tree - Size Implementation

```
public class BinaryTree
{
    private TreeNode root;
    ...
    public void size()
    {
        size(root);
    }

    private void size(TreeNode current)
    {
        // base case
        if (current == null)
            return 0;

        // recursive case
        int left = size(current.left);
        int right = size(current.right);
        return 1 + left + right; // count 1 for current
    }

    class TreeNode { ... }
}
```

Questions & Answer


Thank You!